UNIT 3 – INHERITANCE AND POLYMORPHISM

1. INHERITANCE IN JAVA

1.1 MEANING OF INHERITANCE

Inheritance is an important feature of Object-Oriented Programming in Java. It allows one class to acquire the properties and methods of another class. The class that provides the properties and methods is called the **parent or superclass**, while the class that receives them is called the **child or subclass**.

In simple words:

Inheritance means creating a new class by reusing the features of an existing class.

REAL-LIFE EXAMPLE

Suppose `Animal` has methods such as `eat()` and `sleep()`. A `Dog` is an animal, so the `Dog` class can inherit these common methods from `Animal`.

```
   Animal
(Parent Class)
      |
      ↓
    Dog
(Child Class)
```

1.2 WHY DO WE USE INHERITANCE?

Inheritance is mainly used for **code reusability**. Common properties and methods can be written once in the parent class and reused by multiple child classes.

It reduces code duplication and makes programs easier to maintain. It also provides the foundation for concepts such as **method overriding and runtime polymorphism**.

ADVANTAGES

- Code reusability
- Reduces code duplication
- Easy maintenance
- Supports polymorphism
- Creates a relationship between classes

1.3 EXTENDS KEYWORD

The **`extends` keyword** is used when one class inherits another class. The class after `extends` is the **parent/superclass**, and the class before `extends` is the **child/subclass**.

SYNTAX

```
class Child extends Parent
{
    // child class members
}
```

EXAMPLE

```
class Animal
{
    void eat()
    {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog barks");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Dog d = new Dog();

        d.eat();
        d.bark();
    }
}
```

OUTPUT

```
Animal eats
Dog barks
```

Here, `Dog` inherits `eat()` from `Animal`. Therefore, the `Dog` object can access both the inherited `eat()` method and its own `bark()` method.

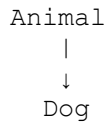
1.4 TYPES OF INHERITANCE

Java inheritance can be classified into different types depending on how the classes are related. The commonly discussed types are **Single, Multilevel, Hierarchical, and Multiple inheritance**.

Java supports the first three using classes. Multiple inheritance is achieved through **interfaces**.

A. SINGLE INHERITANCE

In **single inheritance**, one child class inherits from one parent class. It is the simplest form of inheritance.



EXAMPLE

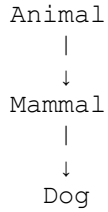
```
class Animal
{
    void eat()
    {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog barks");
    }
}
```

Here, `Dog` inherits the `eat()` method from `Animal`.

B. MULTILEVEL INHERITANCE

In **multilevel inheritance**, inheritance occurs through multiple levels. One class inherits another class, and a third class inherits the second class.



EXAMPLE

```
class Animal
{
    void eat()
    {
```

```
        System.out.println("Eating");
    }
}

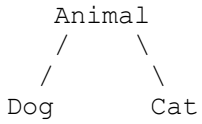
class Mammal extends Animal
{
    void walk()
    {
        System.out.println("Walking");
    }
}

class Dog extends Mammal
{
    void bark()
    {
        System.out.println("Barking");
    }
}
```

A `Dog` object can access `eat()`, `walk()`, and `bark()` because it receives features from both parent levels.

C. HIERARCHICAL INHERITANCE

In **hierarchical inheritance**, multiple child classes inherit from the same parent class. Common functionality can therefore be placed in the parent and reused by different child classes.



EXAMPLE

```
class Animal
{
    void eat()
    {
        System.out.println("Animal eats");
    }
}

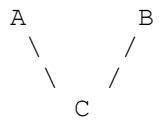
class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal
{
    void meow()
    {
        System.out.println("Cat meows");
    }
}
```

Both `Dog` and `Cat` can use the inherited `eat()` method.

D. MULTIPLE INHERITANCE

Multiple inheritance means that one child class inherits from more than one parent class.



Java does **not support multiple inheritance through classes** because it can create ambiguity when parent classes contain methods with the same name.

For example:

```
class C extends A, B
{
}
```

is not allowed in Java.

E. MULTIPLE INHERITANCE USING INTERFACES

Java supports multiple inheritance through **interfaces**. A class can implement multiple interfaces using the `implements` keyword.

```
interface A
{
    void showA();
}

interface B
{
    void showB();
}

class C implements A, B
{
    public void showA()
    {
        System.out.println("A");
    }

    public void showB()
    {
        System.out.println("B");
    }
}
```

Here, class `C` implements both interfaces `A` and `B`.

2. SUPER CLASS AND SUB CLASS

2.1 SUPER CLASS

A **superclass** is the class whose properties and methods are inherited by another class. It is also called a **parent class** or **base class**.

The superclass contains common features that can be shared by its child classes. This helps us avoid writing the same code repeatedly.

EXAMPLE

```
class Animal
{
    void eat()
    {
        System.out.println("Animal eats");
    }
}
```

Here, `Animal` is the **superclass**.

2.2 SUB CLASS

A **subclass** is a class that inherits properties and methods from another class. It is also called a **child class** or **derived class**.

A subclass can use the inherited members and can also define its own new members. It can additionally override inherited methods when required.

EXAMPLE

```
class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog barks");
    }
}
```

Here, `Dog` is the **subclass** of `Animal`.

2.3 RELATIONSHIP BETWEEN SUPERCLASS AND SUBCLASS

The superclass provides common features, while the subclass reuses and extends those features. The relationship is created using the `extends` keyword.

```
Superclass
  Animal
    |
    ↓
 Subclass
    Dog
```

The subclass can access appropriate inherited members of the superclass and can also contain its own members.

2.4 EXAMPLE OF SUPERCLASS AND SUBCLASS

```
class Animal
{
    void eat()
    {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog barks");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

OUTPUT

```
Animal eats
Dog barks
```

Here:

- `Animal` → Superclass
- `Dog` → Subclass
- `eat()` → Inherited method
- `bark()` → Subclass method

3. METHOD OVERRIDING

3.1 MEANING OF METHOD OVERRIDING

Method overriding occurs when a child class provides its own implementation of a method that is already defined in the parent class.

The method in the child class must have the same method name and compatible parameters as the parent method. It allows the child class to provide behavior specific to itself.

REAL-LIFE EXAMPLE

An `Animal` may have a method called `sound()`.

A `Dog` can override it to produce `"Bark"` and a `Cat` can override it to produce `"Meow"`.

3.2 EXAMPLE OF METHOD OVERRIDING

```
class Animal
{
    void sound()
    {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal
{
    @Override
    void sound()
    {
        System.out.println("Dog barks");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Dog d = new Dog();

        d.sound();
    }
}
```

OUTPUT

```
Dog barks
```

The `Dog` class replaces the inherited implementation of `sound()` with its own implementation.

3.3 RULES OF METHOD OVERRIDING

When overriding a method, certain rules must be followed. The method in the child class should have the same name and compatible parameter list as the parent method.

The access level cannot be more restrictive than the overridden method. Methods such as `final` cannot be overridden, and `static` methods are hidden rather than overridden.

IMPORTANT RULES

1. Method name should be the same.
2. Parameters should match.
3. Return type should be compatible.
4. Access level cannot be more restrictive.
5. `final` methods cannot be overridden.
6. Constructors cannot be overridden.

3.4 `@OVERRIDE` ANNOTATION

`@Override` is an annotation used to tell the compiler that a method is intended to override a parent class method.

It is not compulsory, but it is strongly recommended because the compiler can detect mistakes. For example, if the method name or parameters are incorrect, the compiler can report an error.

EXAMPLE

```
class Animal
{
    void sound()
    {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal
{
    @Override
    void sound()
    {
        System.out.println("Dog barks");
    }
}
```

Here, `@Override` indicates that `Dog` is overriding the `sound()` method of `Animal`.

3.5 RUNTIME METHOD SELECTION

When a method is overridden, Java can decide which implementation to execute at **runtime** based on the actual object.

This is one of the important features of **runtime polymorphism**. The method that belongs to the actual object is selected rather than simply using the reference type.

Example:

```
Animal a = new Dog();

a.sound();
```

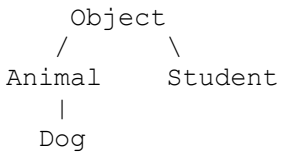
If `Dog` overrides `sound()`, the `Dog` version is executed.

4. OBJECT CLASS

4.1 WHAT IS `OBJECT` CLASS?

`Object` is the **root class of the Java class hierarchy**. Every Java class directly or indirectly inherits from `Object` unless it is a special primitive type.

Therefore, methods defined in `Object` are available to Java objects, subject to normal inheritance and overriding rules.



4.2 `TOSTRING()`

The `toString()` method returns a **string representation of an object**. It is commonly used when we want to display information about an object as text.

By default, it provides a representation based on the object's class and identity. We can override it to return meaningful information.

EXAMPLE

```
class Student
{
    int id = 101;
    String name = "Ravi";

    public String toString()
    {
        return id + " " + name;
    }
}

class Main
{
    public static void main(String[] args)
    {
        Student s = new Student();

        System.out.println(s);
    }
}
```

OUTPUT

```
101 Ravi
```

Here, `System.out.println(s)` automatically uses the `toString()` method.

4.3 `EQUALS()`

The `equals()` method is used to compare objects for **logical equality**.

The default implementation in `Object` compares object identity, but classes can override `equals()` to define their own logical equality. For example, a `Student` class can compare students based on their ID.

EXAMPLE

```
class Student
{
    int id;

    Student(int id)
    {
        this.id = id;
    }

    public boolean equals(Object obj)
    {
        Student s = (Student)obj;

        return this.id == s.id;
    }
}
```

Here, two `Student` objects can be considered equal when their IDs are equal.

4.4 `HASHCODE()`

The `hashCode()` method returns an integer value associated with an object. It is especially important when objects are used in hash-based collections such as `HashSet` and `HashMap`.

If two objects are considered equal according to `equals()`, their `hashCode()` values must also be equal. Therefore, when overriding `equals()`, `hashCode()` should normally be overridden consistently.

EXAMPLE

```
class Student
{
    int id;

    Student(int id)
    {
        this.id = id;
    }

    public int hashCode()
    {
        return id;
    }
}
```

Here, the student's ID is used to generate the hash code.

4.5 GETCLASS()

The `getClass()` method returns information about the **runtime class of an object**.

It is useful when we need to determine which class an object actually belongs to during program execution. The method returns a `Class` object representing the object's runtime type.

EXAMPLE

```
class Student
{
}

class Main
{
    public static void main(String[] args)
    {
        Student s = new Student();

        System.out.println(s.getClass());
    }
}
```

OUTPUT

```
class Student
```

Here, `getClass()` tells us that the object belongs to the `Student` class.

4.6 COMMON METHODS OF OBJECT CLASS

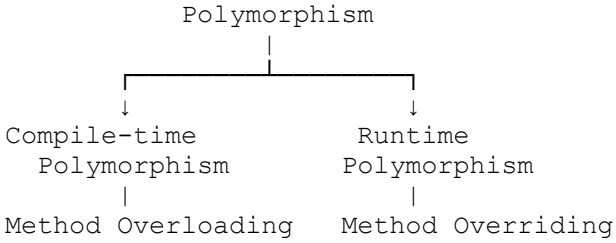
OD	OSE
ing()	s string representation of an object
s()	res objects for logical equality
ode()	s hash code of an object
ass()	s runtime class information

5. POLYMORPHISM

5.1 MEANING OF POLYMORPHISM

Polymorphism is an important concept of Object-Oriented Programming that means **"many forms."**

It allows the same method name or reference to behave differently in different situations. In Java, polymorphism is mainly divided into **compile-time polymorphism** and **runtime polymorphism**.



5.2 COMPILE-TIME POLYMORPHISM

Compile-time polymorphism occurs when the compiler decides which method should be called during compilation.

In Java, **method overloading** is the main example of compile-time polymorphism. Multiple methods can have the same name but must have different parameter lists.

EXAMPLE

```
class Calculator
{
    int add(int a, int b)
    {
        return a + b;
    }

    int add(int a, int b, int c)
    {
        return a + b + c;
    }
}

class Main
{
    public static void main(String[] args)
    {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
        System.out.println(c.add(10, 20, 30));
    }
}
```

OUTPUT

```
30
60
```

The compiler determines which `add()` method should be called based on the number of arguments.

5.3 METHOD OVERLOADING

Method overloading means defining multiple methods with the **same name but different parameter lists** in the same class.

The parameters can differ in their number, type, or order. Changing only the return type is not sufficient for method overloading.

EXAMPLE

```
class Calculator
{
    void add(int a, int b)
    {
        System.out.println(a + b);
    }

    void add(double a, double b)
    {
        System.out.println(a + b);
    }
}
```

Here, both methods are named `add()`, but their parameter types are different.

5.4 TYPES OF METHOD OVERLOADING

A. DIFFERENT NUMBER OF PARAMETERS

```
void add(int a, int b)
{
}

void add(int a, int b, int c)
{
}
```

The methods have the same name but different numbers of parameters.

B. DIFFERENT PARAMETER TYPES

```
void show(int a)
{
}

void show(String a)
{
}
```

Here, one method accepts an `int`, while the other accepts a `String`.

C. DIFFERENT ORDER OF PARAMETERS

```
void display(int a, String b)
{
}

void display(String a, int b)
{
}
```

The methods have the same number of parameters, but the parameter order is different.

5.5 RUNTIME POLYMORPHISM

Runtime polymorphism occurs when the method to execute is determined during **program execution**.

It is achieved through **method overriding and inheritance**. A parent class reference can refer to a child class object, and the overridden method of the actual child object is executed.

EXAMPLE

```
class Animal
{
    void sound()
    {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal
{
    @Override
    void sound()
    {
        System.out.println("Dog barks");
    }
}

class Main
{
    public static void main(String[] args)
    {
        Animal a = new Dog();

        a.sound();
    }
}
```

OUTPUT

```
Dog barks
```

Although the reference is of type `Animal`, the actual object is `Dog`. Therefore, the overridden `Dog` method is executed.

5.6 METHOD OVERLOADING VS METHOD OVERRIDING

FEATURE	METHOD OVERLOADING	METHOD OVERRIDING
Polymorphism	Compile-time	Runtime
Inheritance required?	No	Yes
Method name	Same	Same
Parameters	Must be different	Must match
Decision	Compile time	Runtime
Main purpose	Multiple forms of a method	Change inherited behavior

6. DYNAMIC BINDING

6.1 MEANING OF DYNAMIC BINDING

Dynamic binding means that the method to be executed is decided at **runtime**, rather than at compile time. It mainly occurs when a parent class reference refers to a child class object.

When a method is overridden, Java checks the actual object at runtime and calls the appropriate overridden method. Therefore, dynamic binding is closely connected with **runtime polymorphism**.

SIMPLE EXAMPLE

```
class Animal
{
    void sound()
    {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal
{
    @Override
    void sound()
    {
        System.out.println("Dog barks");
    }
```

```
}

class Main
{
    public static void main(String[] args)
    {
        Animal a = new Dog();

        a.sound();
    }
}
```

OUTPUT

```
Dog barks
```

WHY?

```
Animal a = new Dog();
```

Here:

- Reference type → `Animal`
- Actual object → `Dog`

At runtime, Java sees that the object is a `Dog`, so it calls the `Dog` version of `sound()`.

6.2 RUNTIME BINDING

Runtime binding means connecting a method call with its actual method implementation during program execution.

It is commonly seen when a superclass reference points to a subclass object. If the method is overridden, Java selects the subclass implementation at runtime.

```
Animal a = new Dog();

        ↓
Reference → Animal
Object    → Dog

        ↓

a.sound()

        ↓

Dog's sound() is executed
```

6.3 LATE BINDING

Dynamic binding is also called **late binding** because the decision about which overridden method to execute is made late, during runtime.

The compiler checks whether the method call is valid, but the JVM determines the actual overridden implementation when the program runs.

Therefore:

Late binding = Method selection at runtime

6.4 RELATIONSHIP BETWEEN OVERRIDING AND DYNAMIC BINDING

Method overriding creates multiple possible implementations of the same method. Dynamic binding determines which implementation should execute when the program runs.

Therefore, **method overriding provides the different implementations**, while **dynamic binding selects the appropriate implementation at runtime**.

```
Inheritance
     ↓
Method Overriding
     ↓
Parent Reference → Child Object
     ↓
Dynamic Binding
     ↓
Runtime Polymorphism
```

7. GENERIC PROGRAMMING

7.1 MEANING OF GENERIC PROGRAMMING

Generic programming allows us to write classes and methods that can work with different types of data. Java provides **generics** using type parameters such as `<T>`.

Instead of creating separate classes or methods for `Integer`, `String`, `Double`, etc., we can create one reusable generic implementation.

EXAMPLE

```
class Box<T>
{
    T value;

    void set(T value)
    {
        this.value = value;
    }

    T get()
    {
        return value;
    }
}
```

Here, `T` represents a type that will be specified when the object is created.

7.2 GENERICS

Generics allow us to specify the type of data that a class, interface, or method can work with.

They provide **type safety** and reduce the need for explicit type casting. Generics are commonly used with Java collections such as `ArrayList<T>` and `HashMap<K,V>`.

EXAMPLE

```
ArrayList<String> names = new ArrayList<>();
```

Here:

```
String → Type of data stored
```

So the list is designed to store `String` values.

7.3 GENERIC CLASS

A **generic class** is a class that uses a type parameter. The actual data type is specified when an object of the class is created.

EXAMPLE

```
class Box<T>
{
    T value;

    Box(T value)
    {
        this.value = value;
    }

    void display()
    {
        System.out.println(value);
    }
}

class Main
{
    public static void main(String[] args)
    {
        Box<Integer> b1 = new Box<>(100);
        Box<String> b2 = new Box<>("Java");

        b1.display();
        b2.display();
    }
}
```

OUTPUT

```
100
```

```
Java
```

The same `Box` class works with both `Integer` and `String`.

7.4 GENERIC METHOD

A **generic method** is a method that can work with different types of data. The type parameter is declared before the return type of the method.

EXAMPLE

```
class Display
{
    static <T> void show(T value)
    {
        System.out.println(value);
    }

    public static void main(String[] args)
    {
        show(100);
        show("Hello");
        show(10.5);
    }
}
```

OUTPUT

```
100
Hello
10.5
```

The same method can accept different data types.

7.5 TYPE SAFETY

Generics provide **type safety**, which means that the compiler can prevent incompatible data from being inserted into a generic collection or class.

For example:

```
ArrayList<String> names = new ArrayList<>();

names.add("Ravi");
names.add("Kiran");
```

This is allowed because both values are `String`.

But:

```
names.add(100);
```

will produce a compilation error because `100` is an `Integer`, not a `String`.

IMPORTANT POINT

Generics help detect type-related errors during compilation instead of runtime.

8. CASTING OBJECTS

8.1 MEANING OF OBJECT CASTING

Object casting means converting a reference from one object type to another related type.

Casting is generally used when objects are connected through inheritance. Java provides two important types of object casting: **upcasting and downcasting**.

```
Object Casting
       |
       ├── Upcasting
       |
       └── Downcasting
```

8.2 UPCASTING

Upcasting means converting a child class reference into a parent class reference.

It is generally performed automatically because every child object is also an object of its parent type.

EXAMPLE

```
class Animal
{
    void eat()
    {
        System.out.println("Animal eats");
    }
}
class Dog extends Animal
{
    void bark()
    {
        System.out.println("Dog barks");
    }
}
class Main
{
    public static void main(String[] args)
    {
        Animal a = new Dog();
        a.eat();
    }
}
```

Here:

```
Animal a = new Dog();
```

is **upcasting**.

```
Dog object
    ↓
Animal reference
```

8.3 DOWNCASTING

Downcasting means converting a parent class reference back into a child class reference.

Unlike upcasting, downcasting must generally be written explicitly using the cast operator.

EXAMPLE

```
Animal a = new Dog();

Dog d = (Dog)a;

d.bark();
```

Here:

```
Dog d = (Dog)a;
```

converts the `Animal` reference into a `Dog` reference.

8.4 EXPLICIT OBJECT CASTING

When downcasting, we use the class name inside parentheses.

SYNTAX

```
Child obj = (Child) parentReference;
```

EXAMPLE

```
Animal a = new Dog();

Dog d = (Dog)a;
```

The `(Dog)` tells Java that we want to treat the reference as a `Dog`.

However, the actual object must really be compatible with `Dog`. Otherwise, Java can throw a **`ClassCastException`** at runtime.

9. INSTANCEOF OPERATOR

9.1 MEANING OF INSTANCEOF

The **`instanceof` operator** is used to check whether an object belongs to a particular class or interface type. It returns either `true` or `false`. It is especially useful before performing a downcast.

SYNTAX

```
object instanceof ClassName
```

9.2 EXAMPLE

```
class Animal
{
}
class Dog extends Animal
{
}
class Main
{
    public static void main(String[] args)
    {
        Dog d = new Dog();
        System.out.println(d instanceof Dog);
        System.out.println(d instanceof Animal);
    }
}
```

OUTPUT

```
true
true
```

The first result is `true` because `d` is a `Dog`.

The second result is also `true` because a `Dog` is an `Animal`.

9.3 INSTANCEOF WITH INHERITANCE

The `instanceof` operator follows the inheritance relationship between classes.

If a child object is created, it can also be considered an instance of its parent class.

```
Animal
   ↑
  Dog
```

For a `Dog` object:

```
d instanceof Dog
```

is `true`.

And:

```
d instanceof Animal
```

is also `true`.

9.4 INSTANCEOF BEFORE DOWNCASTING

`instanceof` can be used to safely check whether a reference can be converted to a particular child type.

EXAMPLE

```
Animal a = new Dog();

if(a instanceof Dog)
{
    Dog d = (Dog)a;

    d.bark();
}
```

The condition checks the actual object before performing the downcast.

This helps avoid an invalid cast when the reference does not actually refer to the expected child object.

10. ABSTRACT CLASS

10.1 MEANING OF ABSTRACT CLASS

An **abstract class** is a class declared using the `abstract` keyword. It is designed to act as a base class for other classes and can contain both abstract and concrete methods.

An abstract class cannot be directly instantiated, meaning we cannot normally create an object of it using `new`. Child classes can extend it and provide implementations for its abstract methods.

SYNTAX

```
abstract class Animal
{
    abstract void sound();

    void eat()
    {
        System.out.println("Animal eats");
    }
}
```

10.2 ABSTRACT METHOD

An **abstract method** is a method declared without a body. It specifies what a child class should implement but does not provide the implementation itself.

An abstract method must be declared using the `abstract` keyword. A concrete subclass generally provides the implementation of that method.

EXAMPLE

```
abstract class Animal
{
    abstract void sound();
}
```

Here:

```
abstract void sound();
```

is an abstract method.

10.3 CONCRETE METHOD

A **concrete method** is a normal method that contains a method body and provides an implementation.

An abstract class can contain both abstract methods and concrete methods. Child classes inherit the concrete methods and can implement the abstract methods.

EXAMPLE

```
abstract class Animal
{
    abstract void sound();

    void eat()
    {
        System.out.println("Animal eats");
    }
}
```

Here:

```
sound() → Abstract method
eat()   → Concrete method
```

10.4 ABSTRACT CLASS AND INHERITANCE

Abstract classes are normally used as parent classes. A child class extends the abstract class and provides implementations for its abstract methods.

EXAMPLE

```
abstract class Animal
{
    abstract void sound();

    void eat()
    {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal
{
    void sound()
    {
        System.out.println("Dog barks");
    }
}
```

Here, `Dog` inherits `eat()` and provides an implementation for `sound()`.

10.5 CANNOT DIRECTLY CREATE AN OBJECT

We cannot directly create an object of an abstract class.

For example:

```
Animal a = new Animal();
```

is not allowed because `Animal` is abstract.

Instead, we create an object of a concrete child class:

```
Animal a = new Dog();
```

This is possible because `Dog` provides the required implementation.

11. INTERFACE IN JAVA

11.1 MEANING OF INTERFACE

An **interface** is a Java type used to define a contract that implementing classes must follow. It is useful for achieving abstraction and supports multiple inheritance of type because a class can implement multiple interfaces. Interfaces can contain abstract methods and, depending on Java version and design, also default and static methods.

11.2 CREATING AN INTERFACE

An interface is created using the **`interface` keyword**.

SYNTAX

```
interface InterfaceName
{
    // methods
}
```

EXAMPLE

```
interface Animal
{
    void sound();
}
```

Here, `Animal` is an interface containing the method `sound()`.

11.3 `IMPLEMENTS` KEYWORD

A class uses the **`implements` keyword** to implement an interface.

The implementing class must provide implementations for the required abstract methods.

EXAMPLE

```
interface Animal
{
    void sound();
}
class Dog implements Animal
{
    public void sound()
    {
        System.out.println("Dog barks");
    }
}
```

Here, `Dog` implements the `Animal` interface and provides the implementation of `sound()`.

11.4 IMPLEMENTING INTERFACE METHODS

When a class implements an interface, it must implement the interface's abstract methods unless the class itself is abstract.

EXAMPLE

```
interface Shape
{
    void draw();
}

class Circle implements Shape
{
    public void draw()
    {
        System.out.println("Drawing Circle");
    }
}
```

The `Circle` class provides the implementation of `draw()`.

11.5 MULTIPLE INTERFACES

A Java class can implement more than one interface. This allows a class to follow multiple contracts and is one way Java supports multiple inheritance of type.

EXAMPLE

```
interface A
{
    void showA();
}

interface B
{
    void showB();
}

class C implements A, B
{
    public void showA()
    {
        System.out.println("A");
    }
    public void showB()
    {
        System.out.println("B");
    }
}
 Here, class C implements both A and B.
      A       B
       \     /
        \   /
          C
```

11.6 MULTIPLE INHERITANCE USING INTERFACES

Java does not allow:

```
class C extends A, B
```

when `A` and `B` are classes.

But Java allows:

```
class C implements A, B
```

when `A` and `B` are interfaces.

Therefore, interfaces provide a mechanism for a class to implement multiple contracts.

12. PACKAGE IN JAVA

12.1 MEANING OF PACKAGE

A **package** is a group of related classes, interfaces, and other Java types. Packages are used to organize Java programs and avoid naming conflicts between classes. They also help control access to classes and members.

EXAMPLES

```
java.util
java.io
java.lang
java.sql
```

12.2 CREATING PACKAGES

We can create our own package by using the **`package` keyword** at the beginning of a Java source file.

EXAMPLE

```
package mypackage;
public class Student
{
    public void display()
    {
        System.out.println("Student class");
    }
}
```

Here, `Student` belongs to the package named `mypackage`.

12.3 `package` KEYWORD

The `package` keyword is used to specify which package a Java class belongs to.

It normally appears at the beginning of the source file, before import declarations and class definitions.

EXAMPLE

```
package college;
```

This tells Java that the class belongs to the `college` package.

12.4 `import` KEYWORD

The **`import` keyword** allows us to use classes from another package without writing their complete package name every time.

EXAMPLE

```
import java.util.Scanner;
```

After importing `Scanner`, we can write:

```
Scanner sc = new Scanner(System.in);
```

instead of using the fully qualified class name.

12.5 USER-DEFINED PACKAGES

A **user-defined package** is a package created by the programmer to organize their own classes.

EXAMPLE

```
package college;

public class Student
{
    public void display()
    {
        System.out.println("Student Information");
    }
}
```

The programmer has created the package:

```
college
```

and placed the `Student` class inside it.

12.6 BUILT-IN PACKAGES

Java provides many predefined packages containing commonly used classes and interfaces.

Some important packages are:

PACKAGE	COMMON USE
`java.lang`	Basic Java classes
`java.util`	Collections and utility classes
`java.io`	Input/output operations
`java.sql`	Database connectivity
`java.net`	Networking

13. `JAVA.UTIL` PACKAGE

13.1 MEANING OF `JAVA.UTIL`

`java.util` is a commonly used Java package that provides many **utility classes and interfaces**.

It contains classes for collections, input handling, data structures, date/time-related utilities, and other common programming tasks. Some frequently used classes include `Scanner`, `ArrayList`, `HashMap`, `HashSet`, and `Collections`.

IMPORTING THE PACKAGE

```
import java.util.*;
```

The `*` means classes from that package can be referred to without writing their full package names.

13.2 `SCANNER`

`Scanner` is a class in `java.util` used to **read input from the keyboard** and other input sources.

It provides methods such as `nextInt()`, `nextDouble()`, and `nextLine()` for reading different types of input.

EXAMPLE

```
import java.util.Scanner;

class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name: ");
        String name = sc.nextLine();
        System.out.println("Hello " + name);
    }
}
```

EXAMPLE OUTPUT

```
Enter your name: Vamshi
Hello Vamshi
```

13.3 ARRAYLIST

`ArrayList` is a resizable list provided by the Java Collections Framework.

Unlike a normal array, its size can grow or shrink dynamically as elements are added or removed. It stores objects and is commonly used when the number of elements is not fixed.

EXAMPLE

```
import java.util.ArrayList;

class Main
{
    public static void main(String[] args)
    {
        ArrayList<String> names = new ArrayList<>();

        names.add("Ravi");
        names.add("Kiran");
        names.add("Arun");

        System.out.println(names);
    }
}
```

OUTPUT

```
[Ravi, Kiran, Arun]
```

13.4 HASHMAP

`HashMap` stores data in **key-value pairs**.

Each key is associated with a value, which makes `HashMap` useful when we want to retrieve data using a unique key. Keys should be unique, while values may be duplicated.

EXAMPLE

```
import java.util.HashMap;

class Main
{
    public static void main(String[] args)
    {
        HashMap<Integer, String> students = new HashMap<>();

        students.put(101, "Ravi");
        students.put(102, "Kiran");

        System.out.println(students);
    }
}
```

OUTPUT

```
{101=Ravi, 102=Kiran}
```

Here:

```
101 → Ravi
102 → Kiran
```

13.5 HASHSET

`HashSet` is a collection that stores **unique elements**.

If we try to add the same element more than once, the duplicate element is not stored again. It is useful when duplicate values should not be allowed.

EXAMPLE

```
import java.util.HashSet;

class Main
{
    public static void main(String[] args)
    {
        HashSet<Integer> numbers = new HashSet<>();

        numbers.add(10);
        numbers.add(20);
        numbers.add(10);

        System.out.println(numbers);
    }
}
```

OUTPUT

```
[10, 20]
```

The second `10` is not added because `HashSet` stores unique elements.

13.6 COLLECTIONS

`Collections` is a utility class in `java.util` that provides useful methods for working with collections.

It provides operations such as sorting, reversing, searching, and finding minimum or maximum elements.

EXAMPLE

```
import java.util.ArrayList;
import java.util.Collections;

class Main
{
    public static void main(String[] args)
    {
        ArrayList<Integer> numbers = new ArrayList<>();
```

```
        numbers.add(30);
        numbers.add(10);
        numbers.add(20);
        Collections.sort(numbers);
        System.out.println(numbers);
    }
}
```

OUTPUT

```
[10, 20, 30]
```

Here, `Collections.sort()` arranges the elements in ascending order.

13.7 COMMON JAVA.UTIL CLASSES

CLASS	PURPOSE
Scanner	Reading input
ArrayList	Dynamic list
HashMap	Key-value data
HashSet	Unique elements
Collections	Collection utility operations
LinkedList	Linked list implementation
Stack	Stack data structure
Queue	Queue-related operations